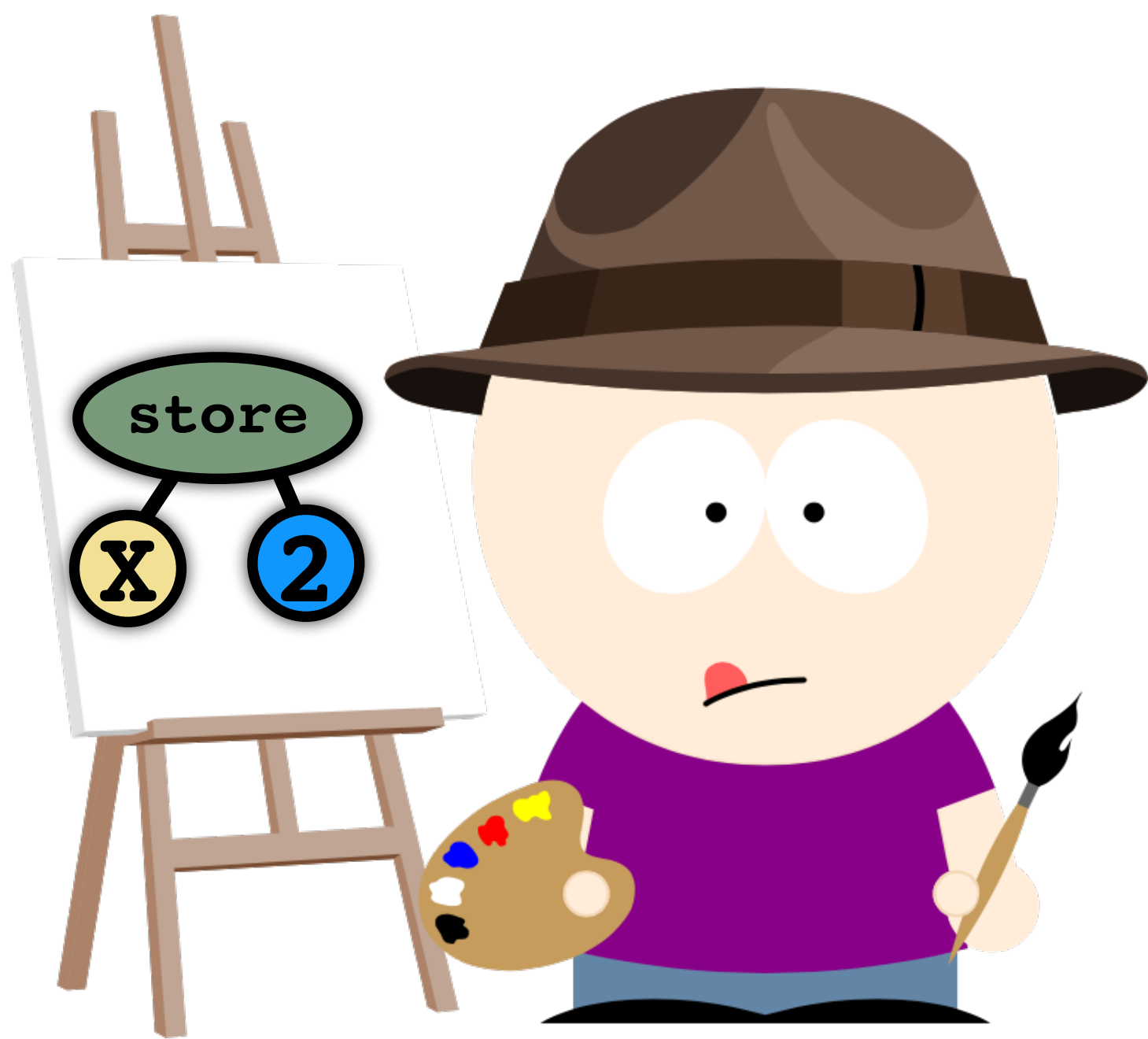




IN-CLASS EXERCISES



INTERMEDIATE CODE 2

—CONTROL FLOW

Download exercises

1. Go to

<https://ligerlabs.org/compilers.html>

2. Download the file

`interm-2-exercises.zip`

3. Open up a terminal and Unzip the file

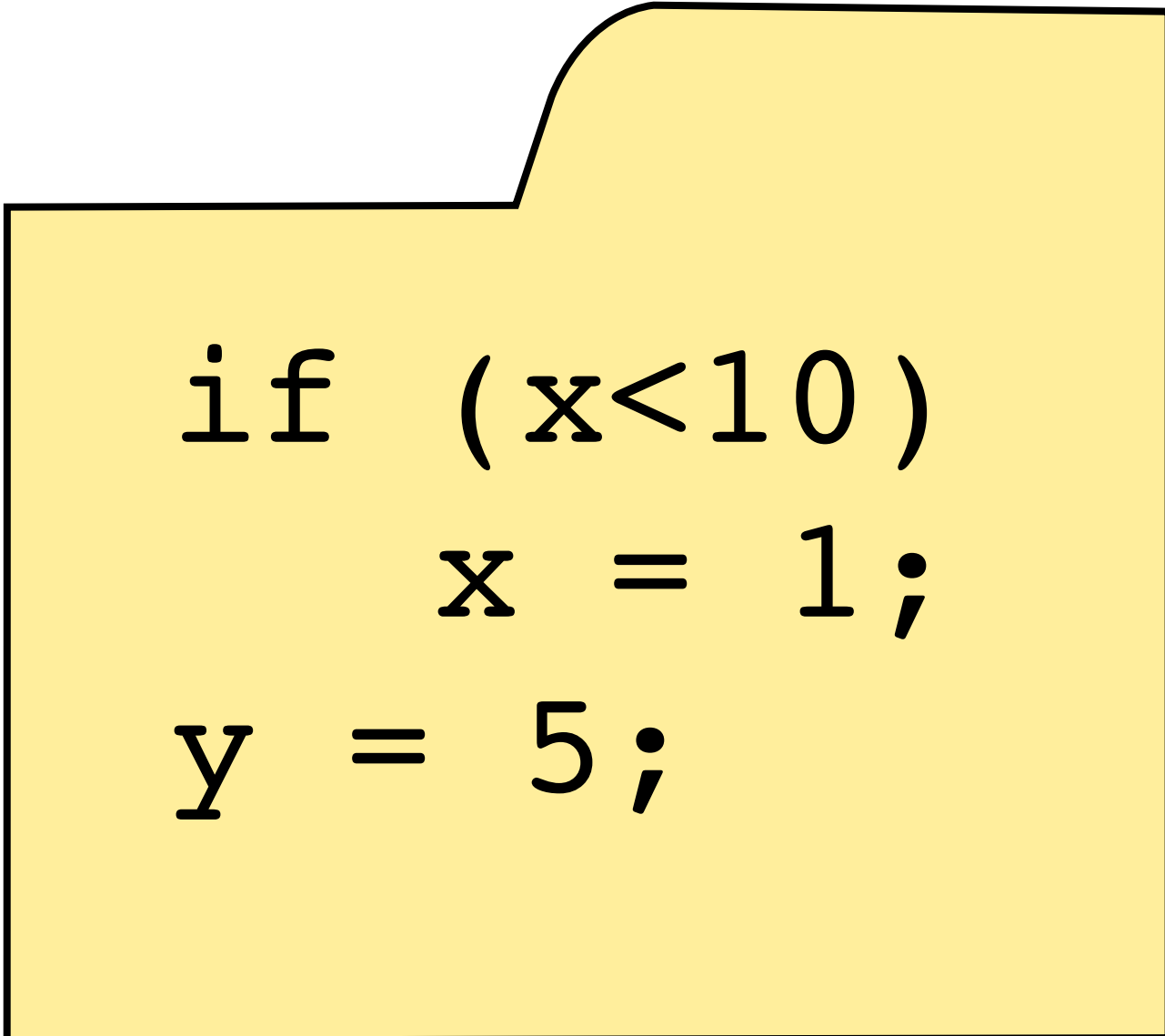
```
> unzip interm-2-exercises.zip
```

```
> cd interm-2-exercises
```

```
> ls
```

Task 1

- Show the quadruple code for the program below if
 1. booleans are **represented numerically**,
 2. booleans are **represented by flow-of-control**.



```
if (x<10 )  
    x = 1;  
y = 5;
```


Task 2

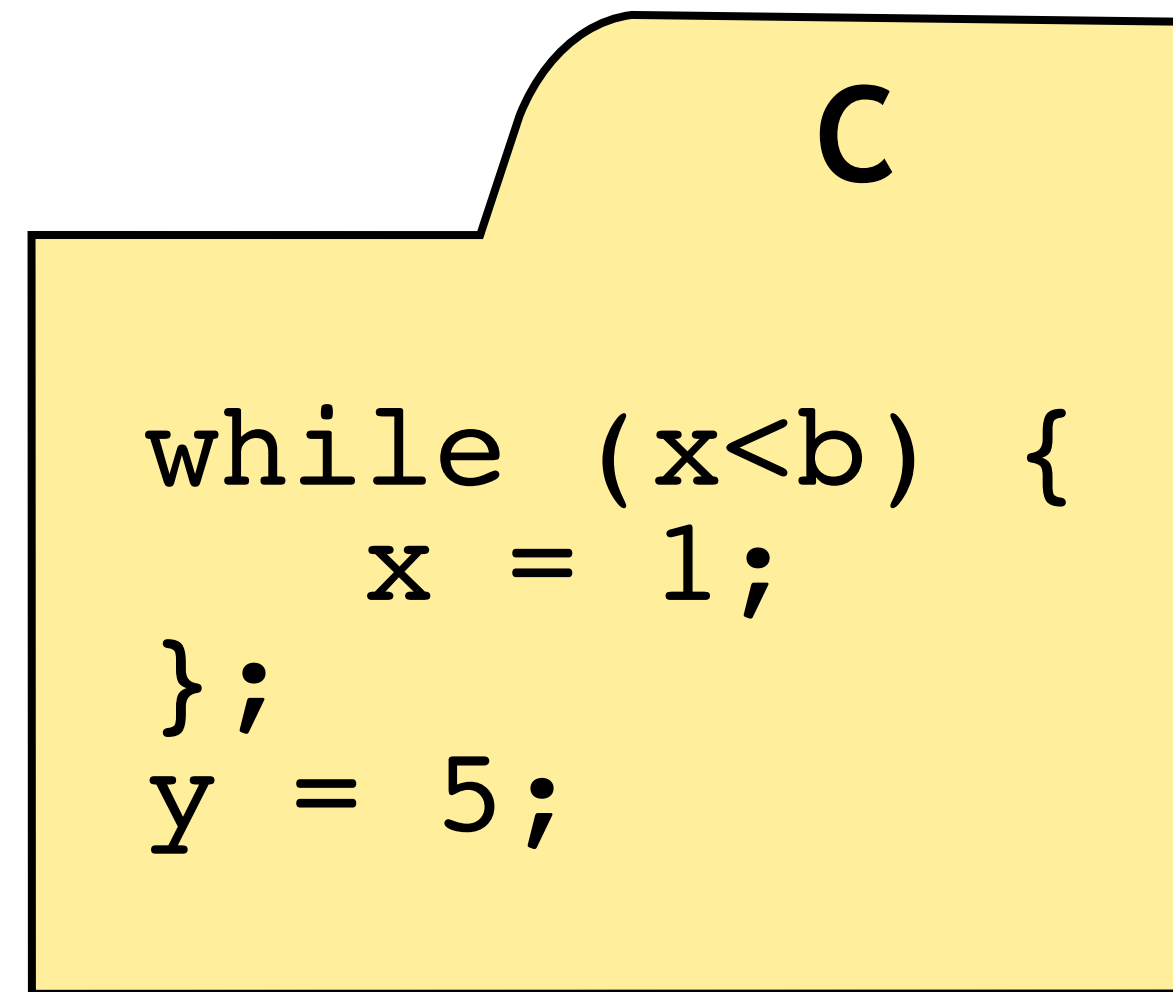
- Show the quadruple code for the programs below if
 1. AND and OR use **eager evaluation** and booleans are **represented numerically**,
 2. AND and OR use **short circuit evaluation** and booleans are **represented by flow-of-control**.

```
if ( (x<10) OR
      (foo())
    x = 1;
y = 5;
```

```
if ( (x<10) AND
      (foo())
    x = 1;
y = 5;
```

Task 3 (a)

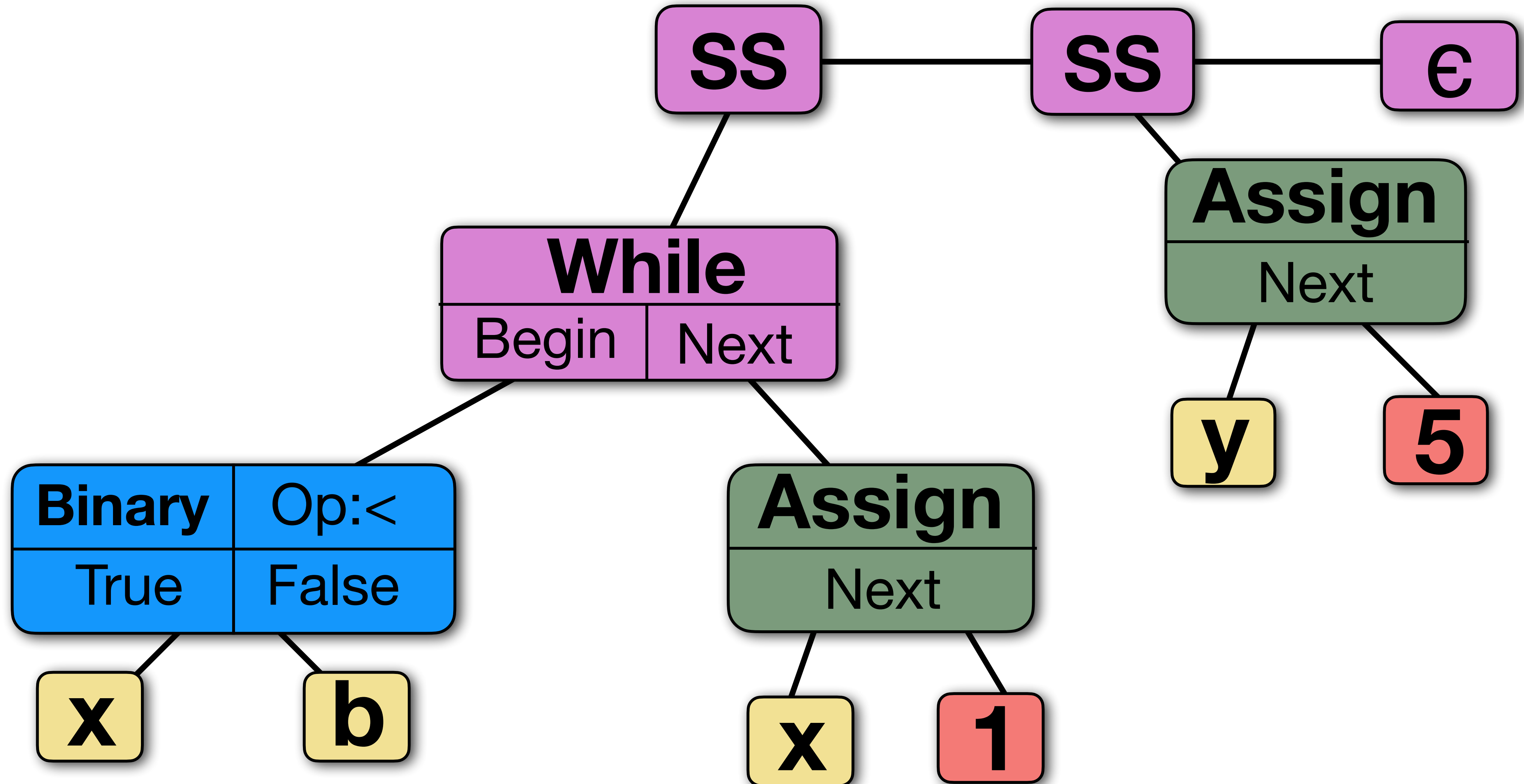
- Consider this program and its AST given on the next slide:



```
while (x<b) {  
    x = 1;  
};  
y = 5;
```

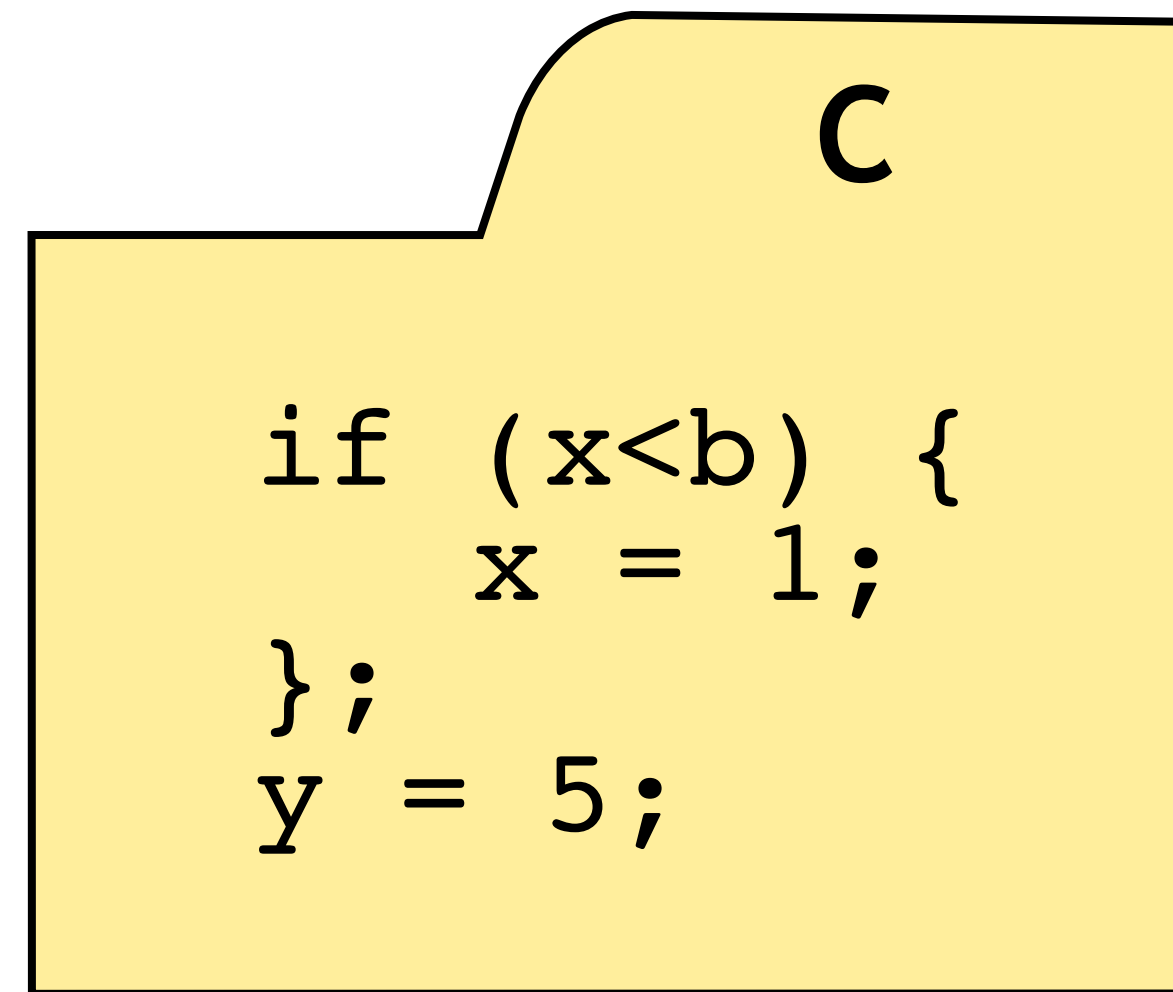
- Show how the attributes `Begin`, `True`, `False`, `Next` are given values as the algorithm generates code for the program.
- Also show the generated quadruple code.

Task 3 (b)



Task 4 (a)

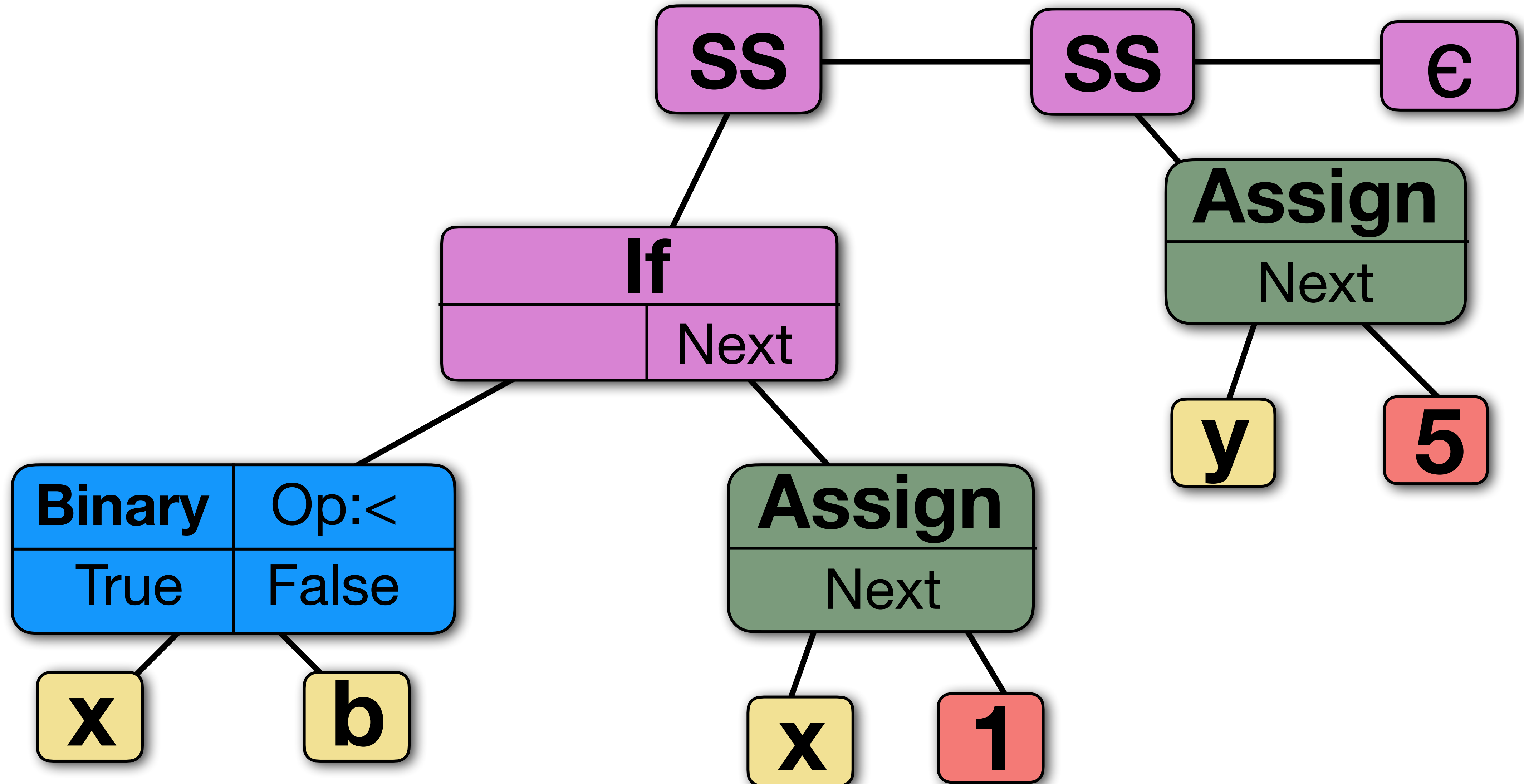
- Consider this program and its AST given on the next slide:



```
if (x < b) {  
    x = 1;  
};  
y = 5;
```

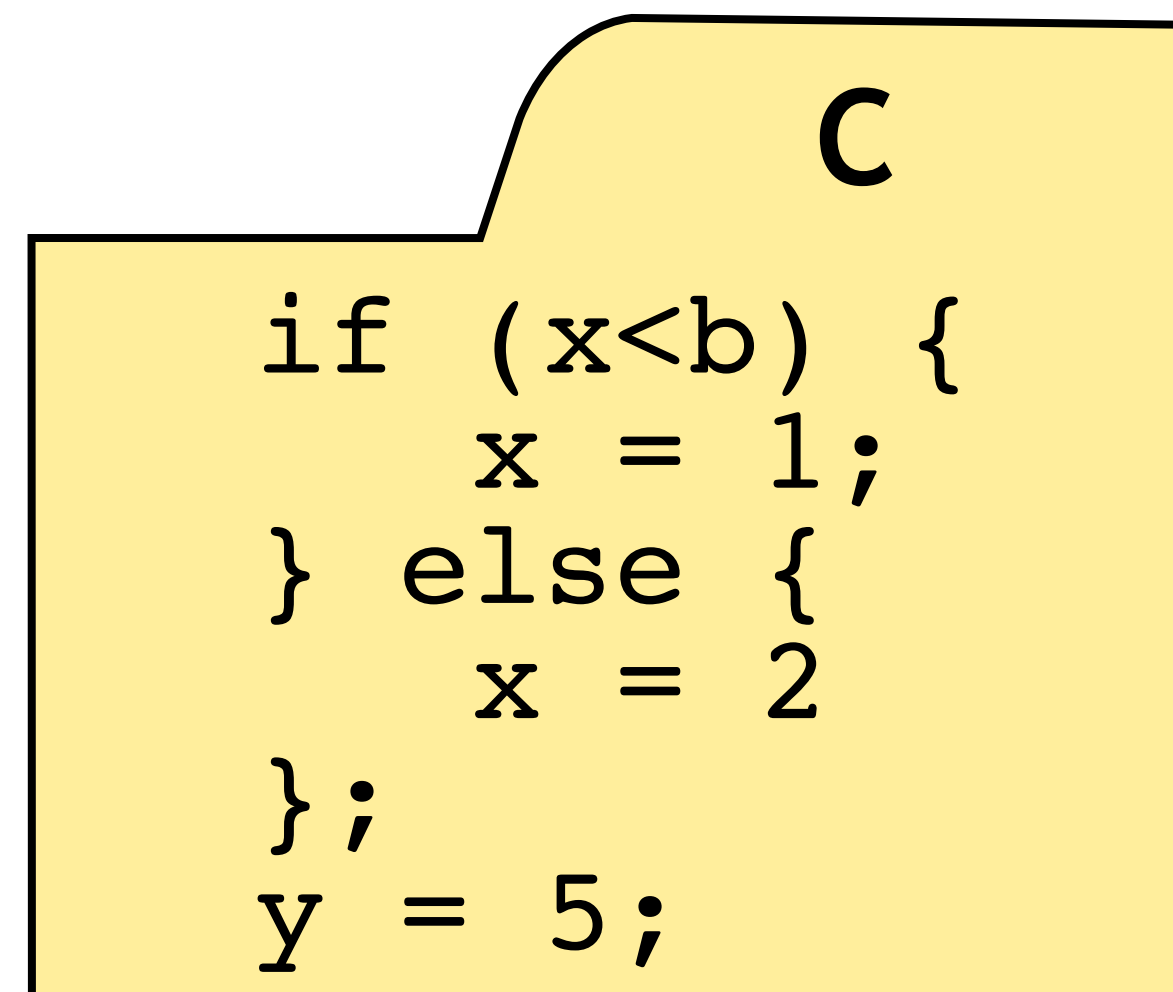
- Show how the attributes `True`, `False`, `Next` are given values as the algorithm generates code for the program.
- Also show the generated quadruple code.

Task 4 (b)



Task 5 (a)

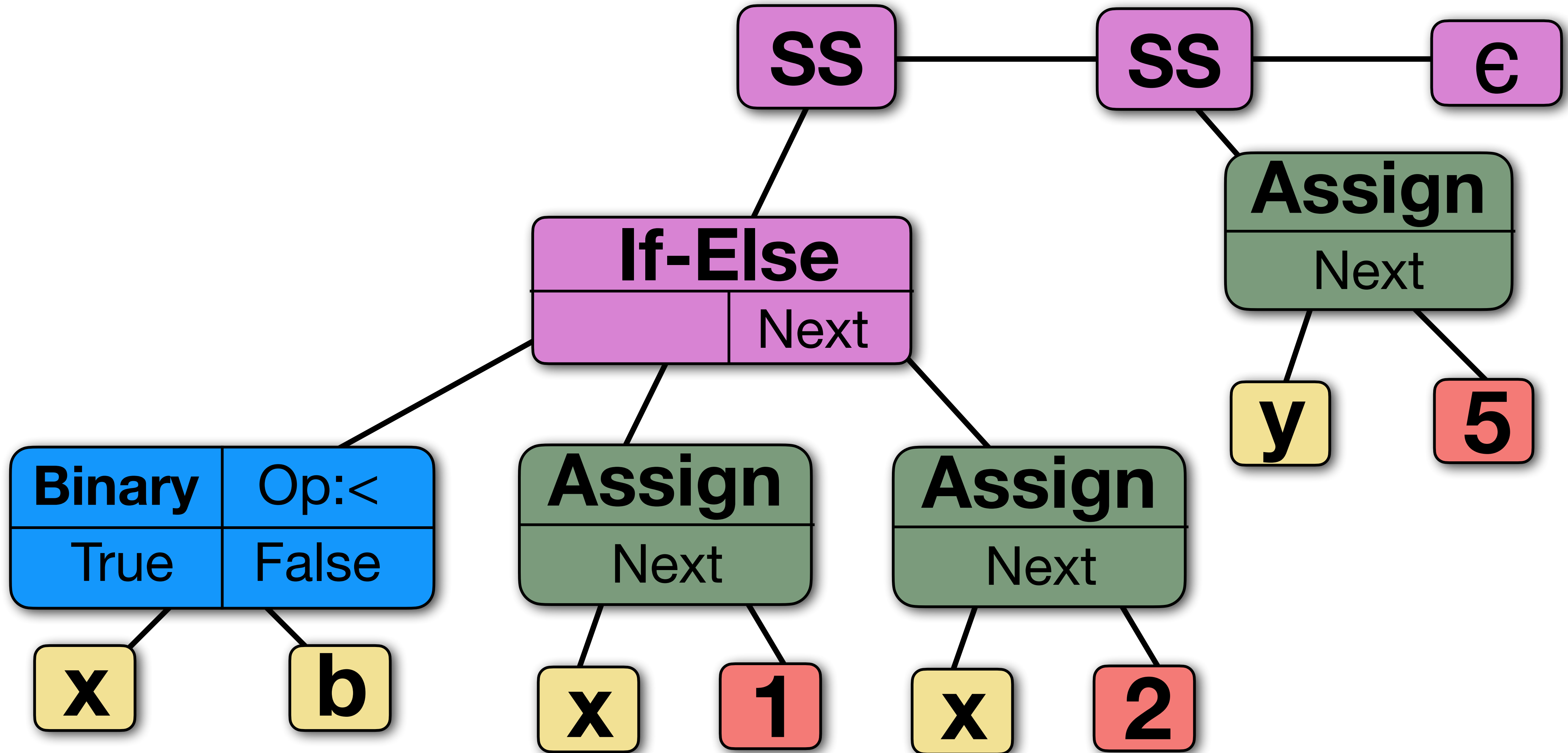
- Consider this program and its AST given on the next slide:



```
C
if (x < b) {
    x = 1;
} else {
    x = 2;
};
y = 5;
```

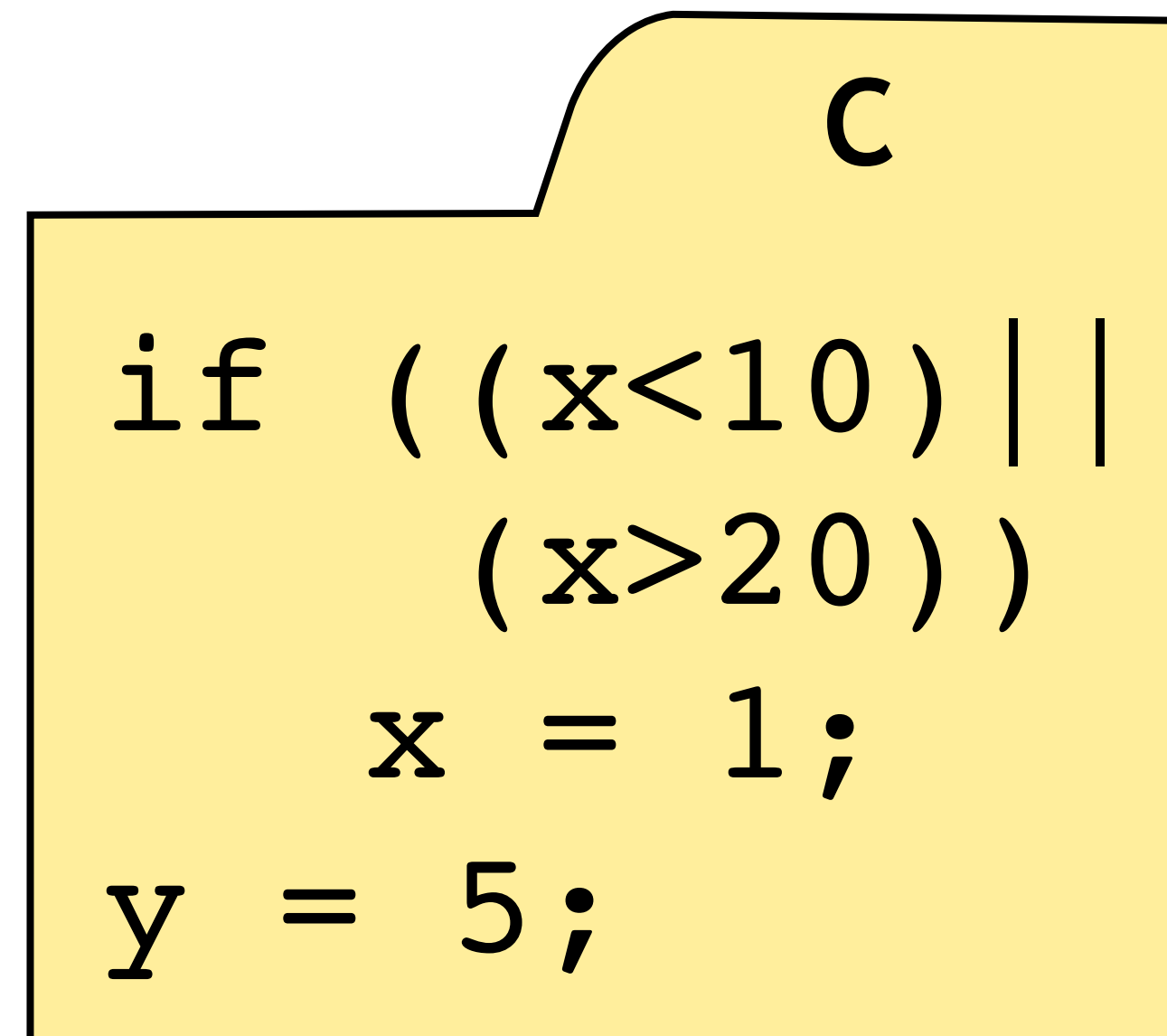
- Show how the attributes `True`, `False`, `Next` are given values as the algorithm generates code for the program.
- Also show the generated quadruple code.

Task 5 (b)



Task 6 (a)

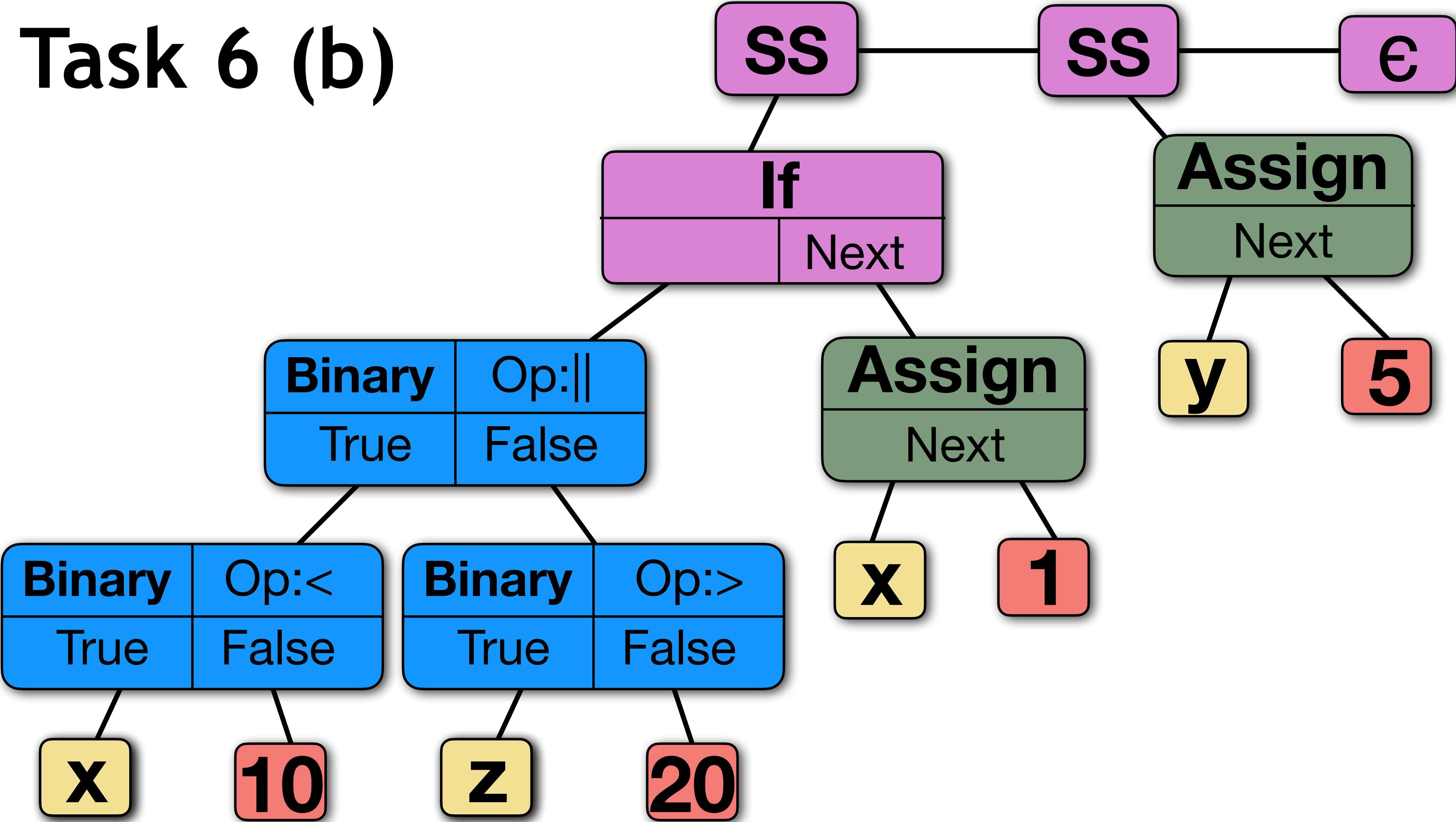
- Consider this program and its AST given on the next slide:



```
C
if ( (x < 10) ||
      (x > 20) )
    x = 1;
y = 5;
```

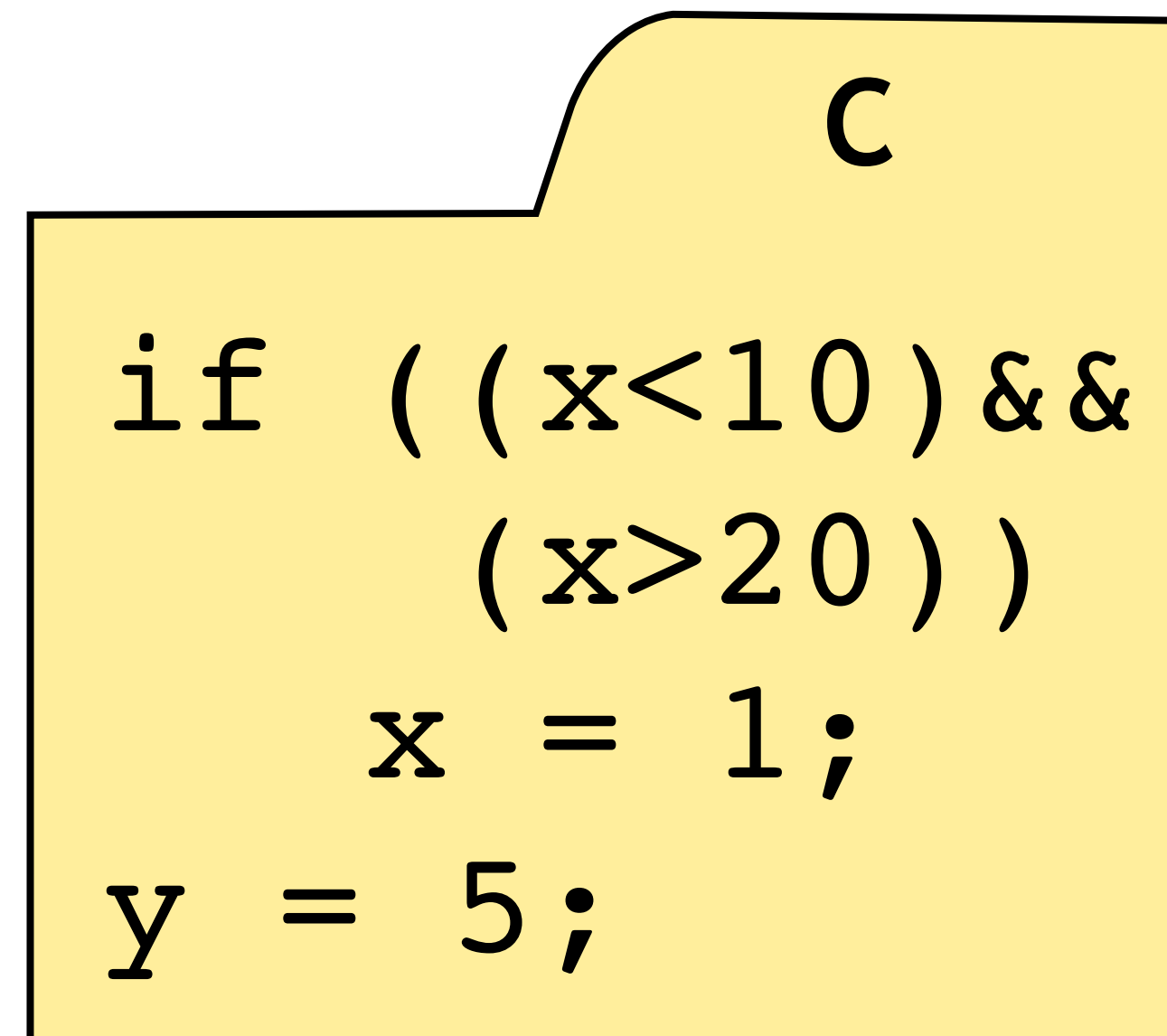
- Show how the attributes `Begin`, `True`, `False`, `Next` are given values as the algorithm generates code for the program.
- Also show the generated quadruple code.

Task 6 (b)



Task 7 (a)

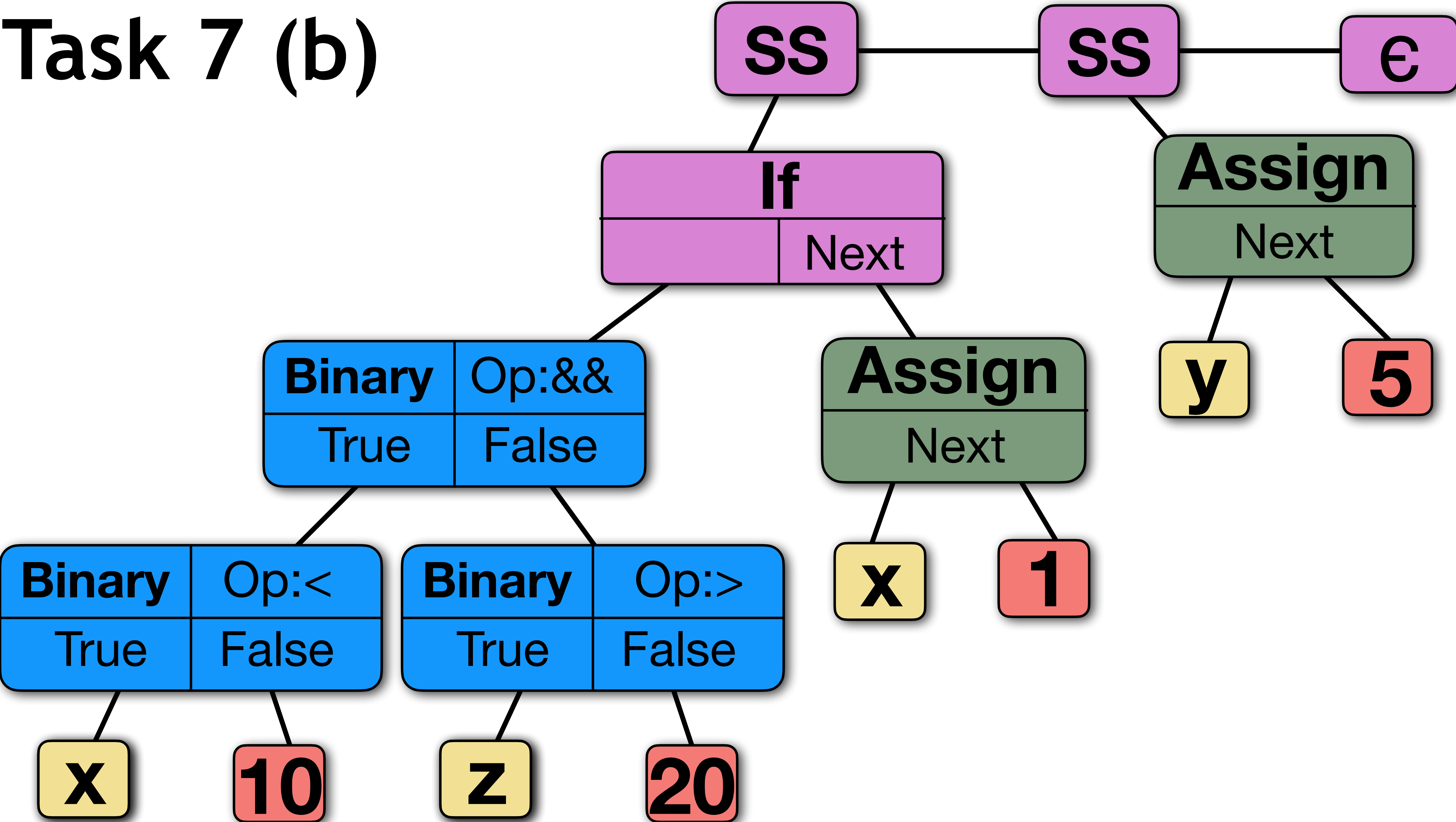
- Consider this program and its AST given on the next slide:



```
C  
  
if ( ( x < 10 ) &&  
      ( x > 20 ) )  
    x = 1;  
y = 5;
```

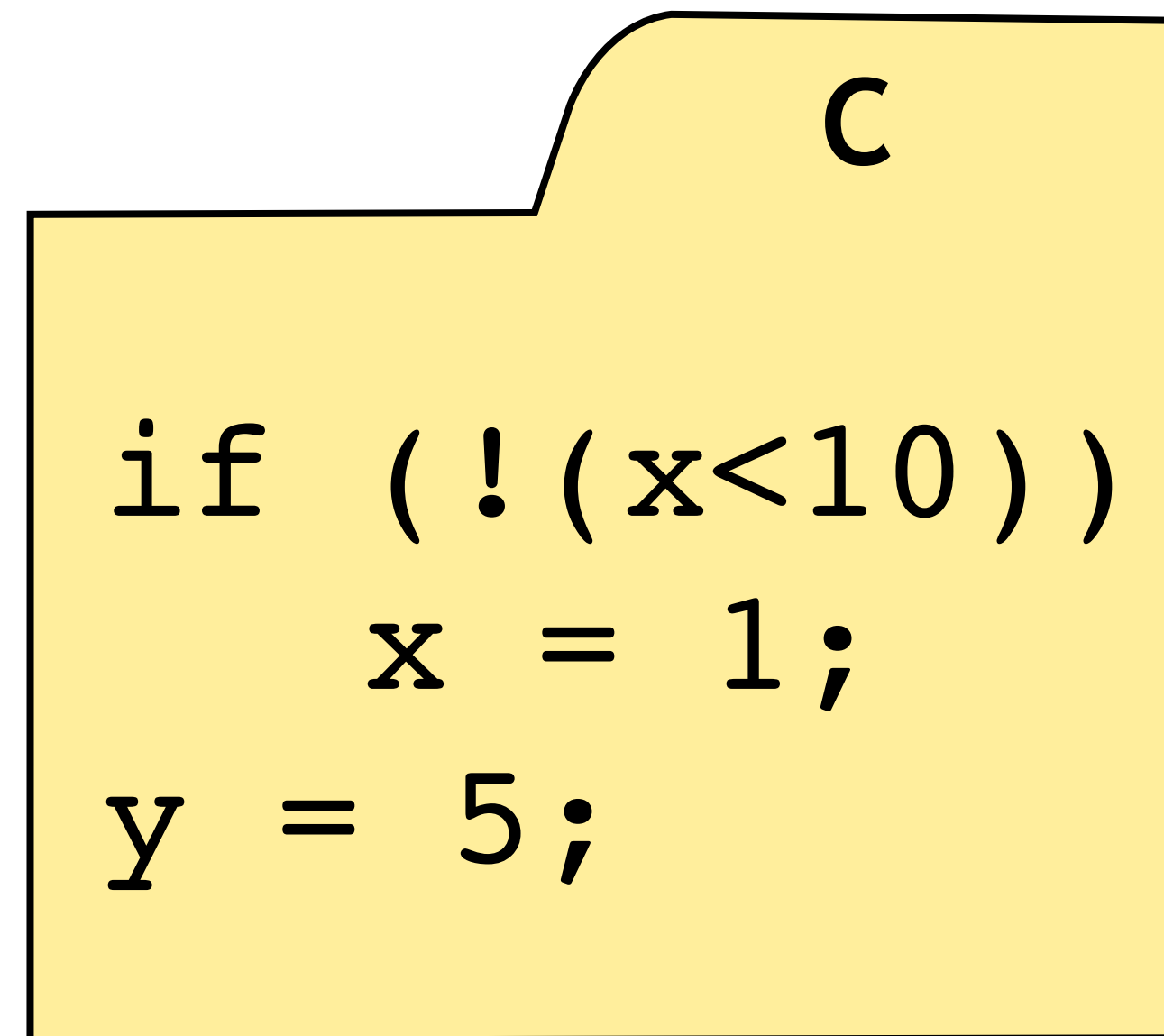
- Show how the attributes `Begin`, `True`, `False`, `Next` are given values as the algorithm generates code for the program.
- Also show the generated quadruple code.

Task 7 (b)



Task 8 (a)

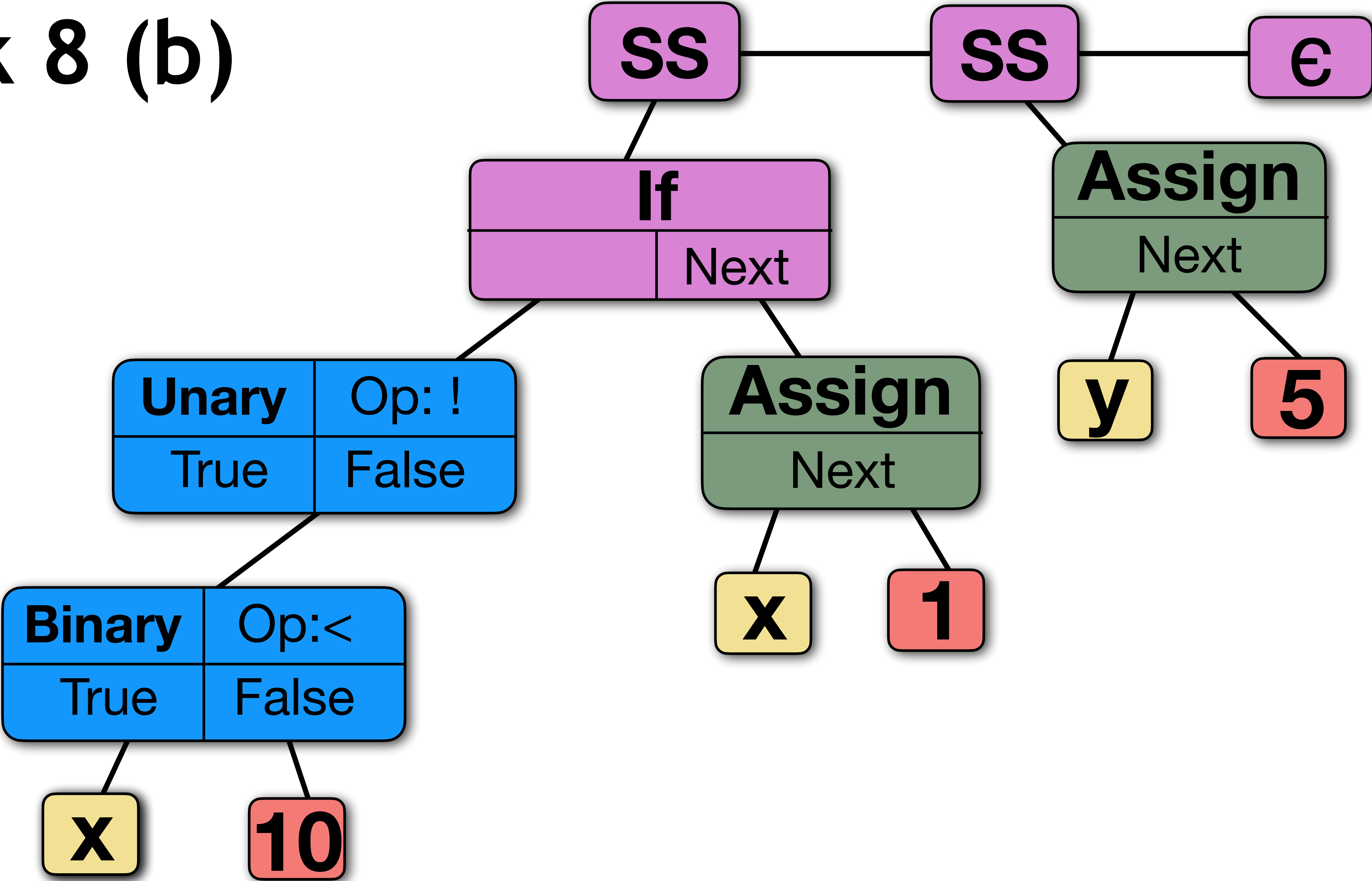
- Consider this program and its AST given on the next slide:



```
if ( ! ( x < 10 ) )  
    x = 1;  
y = 5;
```

- Show how the attributes `True`, `False`, `Next` are given values as the algorithm generates code for the program.
- Also show the generated quadruple code.

Task 8 (b)





DEFINITIONS AND ALGORITHMS

Arithmetic Instructions

Instruction	Semantics
$x \leftarrow y \text{ op } z$	Binary arithmetic or logical operation. Example: Mul, And.
$x \leftarrow \text{op } y$	Unary arithmetic, conversion, or logical operation. Example: Abs, UnaryMinus, Float
$x \leftarrow y$	Copy statement.

Control Flow Instructions

Instruction	Semantics
<code>goto L</code>	Unconditional jump.
<code>if x relop y goto L</code>	Conditional jump. <code>relop</code> is one of <code><</code> , <code>></code> , <code><=</code> , etc. If <code>x relop y</code> evaluates to <code>True</code> , then jump to label <code>L</code> . Otherwise continue with the next tuple. Example: <code>Abs</code> , <code>UnaryMinus</code> , <code>Float</code>
<code>param X;</code> <code>call P, n;</code> <code>V ← returnVal P</code>	1.Make <code>X</code> the next parameter; 2.Make a procedure call to <code>P</code> with <code>n</code> parameters; 3.Assign the return value to <code>V</code>

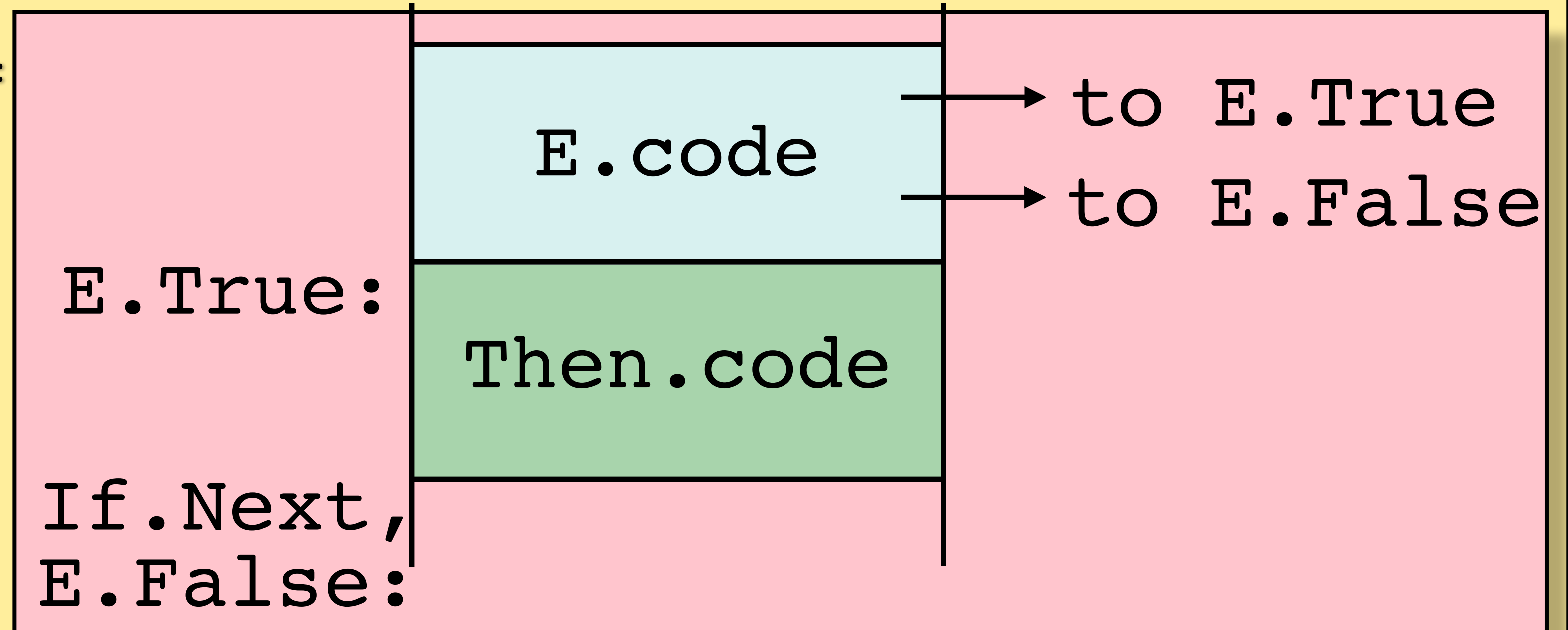
Data Access Instructions

Instruction	Semantics
$x \leftarrow \text{ADDR}(y)$	Address assignment. Set x to the address of y .
$x \leftarrow y[i]$	Indexed load. Set x to the value in the location i memory units beyond y .
$y[i] \leftarrow x$	Indexed store. Set $y[i]$ to the value of x .
$x \leftarrow \text{IND}(y)$	Indirect load. Set x to the value stored at the address in y .
$\text{IND}(x) \leftarrow y$	Indirect store. Set the memory location pointed to by x to the value held by y .

Attribute	Kind	Description
True	Inherited	A label passed into boolean expressions to which the expression should jump if it evaluates to true.
False	Inherited	A label the expression should jump to if false.
Next	Inherited	The label of the statement following the current one.
Begin	Synthesized	The label of the beginning of a while -statement.

IF-Statements

```
If    ::= E:Expr Then:Stat {  
    E.True      := newLabel();  
    E.False     := If.Next;  
    Then.Next   := If.Next;  
    If.Code     :=  
    }  
}
```



While-Statements

```
Wh ::= E:Expr Body:Stat {
```

```
    Wh.Begin    := newLabel();
```

```
    E.True      := newLabel();
```

```
    E.False     := Wh.Next;
```

```
    Body.Next   := Wh.Begin;
```

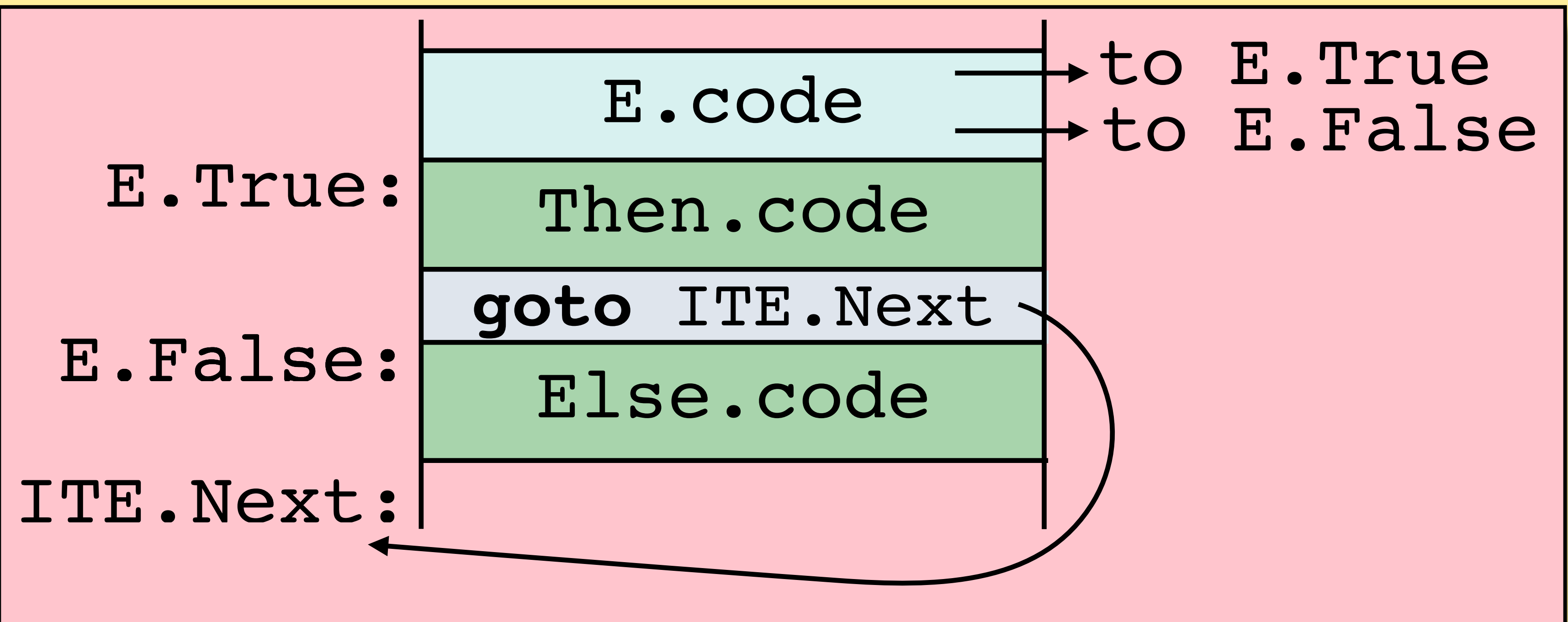
```
    Wh.Code     :=
```

Wh.Begin:	E.code	→ to E.True
E.True:	Body.code	→ to E.False
	goto Wh.Begin	
Wh.Next, E.False:		

```
}
```


If-Then-Else-Statements

```
ITE ::= E:Expr Then:Stat Else:Stat {  
    E.True      := newLabel();  
    E.False     := newLabel();  
    Then.Next   := Else.Next := ITE.Next;  
    ITE.Code    :=
```



```
}
```

Relational Operators

`E ::= L:Expr <Op:String R:Expr {`

`E.Code :=`

```
t1 = L.Code
t2 = R.Code
if (t1 Op t2) goto E.True
goto E.False
```

`}`

NOT

```
E ::= L:Expr {  
  
    L.True  := E.False;  
    L.False := E.True;  
  
}
```


OR

```
E ::= L:Expr R:Expr {  
    L.True := R.True := E.True;  
    L.False := newLabel();  
    R.False := E.False;  
    E.Code :=  
}  
}
```

L.False:

L.code

R.code

to E.True
to L.False
to E.True
to E.False

AND

```
E ::= L:Expr R:Expr {  
  L.False := R.False := E.False;  
  L.True  := newLabel();  
  R.True  := E.True;  
  E.Code  :=  
}  
}
```

L.True:

L.code

R.code

to L.True
to E.False
to E.True
to E.False

COLLBERG.CS.ARIZONA.EDU

LIGERLABS.ORG

SUPPORTED BY
NSF SATC/TTP-1525820
SATC/EDU-2029632

COPYRIGHT © 2024-2026

CHRISTIAN COLLBERG

UNIVERSITY OF ARIZONA